

intensive file systems or database systems is fairly complex. Challenges include data-placement algorithms, cache-management policies, security and maintaining high degrees of fault tolerance.

Enter the NoSQL database, HBase. HBase is being used in production by many companies, including majors like Yahoo, Twitter and Adobe.

StumbleUpon, too, uses it, to crunch the vast amount of data it gets every day. The full list can be found on the HBase Wiki. These companies run Hadoop/HBase clusters, where Hadoop is used for map/reduce jobs, and HBase is used as the production back-end.

HBase caters to the needs of users like social services (adding/updating a huge amount of revisions per day), discovery services and search engines (data crawling and event processing), behavioural targeting of customers, and providing personalisation services (which requires real-time data management and analysis). The HBase community is very active, helpful and diverse. Their IRC channel *#hbase* has HBase developers active on it, which means help is readily available.

## HBase vs RDBMS

In today's market, which has shifted to the Internet as a platform, scalable applications are needed to handle the ever-increasing number of concurrent online users. Current RDBMSs like Oracle and MySQL have a scalability limit. A table of size 100 GB or more becomes harder to scale on such systems, and even using distributed architectures, it takes more time to read data from different instances. Also, in the case of MySQL, migrating data and changing the schema is very difficult. HBase was built specifically to solve these problems.

Though HBase is a database technology, it is very different from RDBMSs. HBase stores data in labelled tables. Tables are dynamically split into regions. Tables contain columns, which are stored in separate files, sparsely over the distributed system. The concepts of rows, columns and attributes in such tables are totally different from

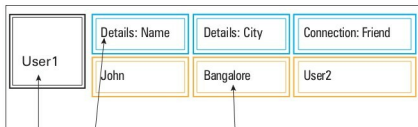


Figure 1: The "Social" table structure

traditional databases. A data row has a sortable key and an arbitrary number of columns. The columns have no predefined width or names. Rows in the same table can have a randomly varying number of columns, if the user likes. Each row entry in the table will contain qualifiers belonging to a family, and having a unique value.

Rows are stored in byte-lexicographical order, and columns are by default sorted within a row. This allows fast and easy scans of data, as well as partial lookups. Also, each row id-column is stored with a time-stamp (thus storing multiple versions), which allows recovery of data that has been messed up by bugs, and also enables detection of collisions and conflicts.

HBase currently supports bytes (the String data-type), and can store ASCII and binary data. The application that is storing and retrieving data is responsible for conversion to/from other data types.

## Installation and usage

Download the latest version of HBase from <http://www.apache.org/dyn/closer.cgi/hadoop/hbase/>.

Simply extract the files from the tarball into a folder of your choice. To start HBase, in a super-user shell, run `./bin/start-hbase`. To get a command-line interface (CLI), run `./hbase shell`. You can verify that HBase is running, and check its status and performance, by visiting <http://localhost:60010> in your browser.

Let's design a database table for a hypothetical social networking site. The data that each user can have associated with it is categorised into details, connections (friends, followers, etc) and notifications (messages, comments). Considering the fact that a new user will have much less data in his profile

than a regular user, using HBase will not only save memory (remember, there are no fixed-width columns taking up space even when empty) but also allow addition of data to the profile later, expanding it without limits.

The conceptual "structure" of our table is depicted in Figure 1.

We need to explain some special terms in HBase parlance, which are used in the figure and elsewhere:

- **Resource:** It can be thought of as a URI/identifier (conceptually similar to a primary key in an RDBMS) for the row, which in our case represents a real-world "User" entity. Each row, as a whole, contains the attributes for the resource.
- **Qualifier:** Also known as a Label, this is an identifiable property of a resource. It has a value associated with it. Qualifiers need not be predefined, and new ones can be added later.
- **Family:** A representative name for a group of qualifiers. Families need to be specified when creating the table; you cannot add families once the table has been created. A family can have any number of qualifiers in it. To create the *Social* table, with the families *Details*, *Connection* and *Notification*, in the CLI, run:

```
create 'Social','Details','Connection','Notification'
```

To insert sample data into this table, run the following queries:

```
put 'Social','User1','Details:Name','John'
put 'Social','User1','Details:City','Bangalore'
put 'Social','User1','Connection:Friend','User2'
put 'Social','User1','Connection:Relative','User3'
put 'Social','User2','Notification:'
```